IES RIBERA DE CASTILLA

PRÁCTICA FINAL

Simulación de Robot Autónomo con Webots

Robótica y Automatización – Nivel Básico

Asignatura:	Sistemas de Aprendizaje Automático + Modelos Inteligencia Artificial
Herramienta:	Webots R2025a
Lenguaje:	Python 3
Duración estimada:	2–3 semanas
Modalidad:	Individual
Calificación máxima:	10 puntos

1. Descripción General del Proyecto

El objetivo de esta práctica es demostrar que el alumnado es capaz de programar un robot virtual en Webots para que navegue de forma autónoma por un entorno con obstáculos, aplicando los conceptos vistos en clase: inicialización del entorno de simulación, lectura de sensores y control de actuadores.

El alumnado no necesita crear el mundo desde cero: se facilitará una escena base con obstáculos estáticos y el robot E-puck (o equivalente). El trabajo consiste en escribir el controlador, documentar la solución y defenderla brevemente.

2. Contexto del Problema

Imagina que trabajas en el equipo de software de una empresa de logística. Necesitáis programar un robot de almacén que, partiendo de un punto inicial, sea capaz de moverse por el almacén sin chocar con las estanterías ni con otros objetos estáticos. El robot debe demostrar que puede moverse de forma segura de manera indefinida.

La simulación en Webots reproduce exactamente este escenario: el robot E-puck actúa como prototipo del robot de almacén y los bloques del entorno simulan las estanterías.

3. Materiales y Configuración Inicial

3.1 Software necesario

- Webots R2025a (descarga gratuita: https://cyberbotics.com)
- Python 3.8+ (ya incluido en Webots o instalado por separado)
- Editor de código: Visual Studio Code, PyCharm o el editor interno de Webots

3.2 Archivos de inicio

El profesor proporcionará los siguientes archivos a través del aula virtual:

- practica_final.wbt – mundo Webots con escenario y robot E-puck precolocados
- mi_controlador.py – plantilla de código vacía con estructura básica comentada
- README.txt – instrucciones de apertura del proyecto en Webots

3.3 Verificación del entorno

Antes de comenzar a programar, el alumnado debe comprobar que:

1. Webots abre el fichero .wbt sin errores.
2. El robot aparece en escena y la simulación puede iniciarse (botón Play).
3. La consola de Webots muestra mensajes del controlador al ejecutar la plantilla.

Consejo de arranque

Si Webots no encuentra el controlador, verifica que la carpeta controllers/ contiene una subcarpeta con el mismo nombre que el controlador asignado al robot en la escena.
En Python, el primer import debe ser: from controller import Robot

4. Requisitos del Controlador

El controlador debe cumplir todos los apartados que se describen a continuación. Cada apartado corresponde directamente a un criterio de la rúbrica.

Apartado A – Inicialización del robot

El controlador debe inicializar correctamente el robot y sus periféricos antes del bucle principal.

- Crear el objeto Robot y obtener el timestep de simulación.
- Obtener referencias a los dos motores (rueda izquierda y rueda derecha) y establecerlos en modo velocidad (setPosition(float('inf'))).
- Obtener referencias a los 8 sensores de proximidad del E-puck (ps0 a ps7) y habilitarlos con el timestep.

Referencia de código – Estructura mínima del controlador

```
from controller import Robot
robot = Robot()
timestep = int(robot.getBasicTimeStep())
left_motor  = robot.getDevice('left wheel motor')
right_motor = robot.getDevice('right wheel motor')
left_motor.setPosition(float('inf'))
right_motor.setPosition(float('inf'))
ps = [robot.getDevice(f'ps{i}') for i in range(8)]
for sensor in ps: sensor.enable(timestep)

while robot.step(timestep) != -1:
    # Tu código de control aquí
    pass
```

Apartado B – Lectura de sensores

Dentro del bucle principal, el controlador debe leer y almacenar los valores de todos los sensores de proximidad en cada iteración.

- Leer los 8 valores con sensor.getValue().
- Identificar cuáles sensores detectan obstáculos con mayor nitidez (ps0, ps1, ps6, ps7 para el frontal; ps2 y ps5 para los laterales).
- Imprimir por consola al menos los valores frontales durante las primeras pruebas para verificar el funcionamiento.

Interpretación de los sensores E-puck

Valor bajo (~70): no hay obstáculo cercano.
Valor medio (70–300): obstáculo a distancia media.
Valor alto (>300): obstáculo muy cercano o contacto inminente.
Umbral sugerido para activar la evasión: 80 (ajústalo según pruebas).

Apartado C – Control de movimiento

El robot debe poder moverse hacia adelante y girar. La velocidad máxima del E-puck es 6.28 rad/s.

- Movimiento hacia adelante: ambos motores a la misma velocidad positiva (ej. 3.0).
- Giro a la derecha: motor izquierdo a velocidad alta, motor derecho a velocidad baja o negativa.
- Giro a la izquierda: motor derecho a velocidad alta, motor izquierdo a velocidad baja o negativa.
- Se recomienda encapsular estos comportamientos en funciones (ir_adelante(), girar_derecha(), etc.).

Apartado D – Comportamiento autónomo: evasión de obstáculos

Este es el núcleo del proyecto. El controlador debe decidir en cada iteración del bucle cómo mover el robot basándose en las lecturas de los sensores.

Se propone el siguiente esquema de comportamiento reactivo (puedes ampliarlo o modificarlo):

4. Si no hay obstáculos (todos los sensores con valor bajo) → avanzar recto.
5. Si hay obstáculo por el lado derecho (ps0, ps1 > umbral) → girar a la izquierda.
6. Si hay obstáculo por el lado izquierdo (ps6, ps7 > umbral) → girar a la derecha.
7. Si hay obstáculo por ambos lados (frontal total) → retroceder y girar.

El robot debe navegar durante al menos 60 segundos continuos sin colisión grave para superar este apartado.

⚠ Atención – Errores frecuentes

No olvides llamar a robot.step(timestep) en cada iteración; sin esto el robot no actualiza los sensores.
Las velocidades fuera del rango [-6.28, 6.28] pueden causar comportamiento inesperado.
Si el robot gira pero no avanza, revisa que ambos motores tienen setPosition(float('inf')) antes del bucle.

Apartado E – Calidad del código

El código entregado debe ser legible y mantenible. Se valorará:

- Nombres de variables descriptivos (ej. velocidad_izquierda, no a o v1).
- Al menos dos funciones definidas con def, además del bucle principal.
- Comentarios que expliquen QUÉ hace cada bloque importante de código y POR QUÉ se tomó esa decisión.
- Sin código «muerto» (bloques comentados que ya no se usan) ni importaciones innecesarias.

5. Informe Escrito (Memoria)

El informe es la documentación del proyecto. Debe estar redactado con claridad y estructura. Extensión máxima: 8 páginas en PDF (sin contar portada).

Estructura obligatoria del informe

8. Portada: nombre(s), fecha, asignatura, título del proyecto.
9. Descripción del problema: explica con tus palabras qué debe hacer el robot.
10. Solución adoptada: describe la lógica de tu controlador (diagrama de flujo opcional pero valorado positivamente).
11. Capturas de pantalla: al menos 3 capturas de la simulación en distintos momentos (robot avanzando, robot evadiendo, robot en zona compleja).
12. Dificultades encontradas: ¿qué no funcionó al principio? ¿Cómo lo resolviste?
13. Conclusiones y mejoras: ¿qué has aprendido? ¿Qué mejorarías con más tiempo?

6. Entregables y Plazos

Entregable	Formato / Dónde entregar	Fecha límite
Código fuente del controlador (.py o .c)	Aula virtual (ZIP con carpeta worlds/ + controllers/)	~ 26 abril 2026
Informe escrito (memoria)	PDF, máx. 8 páginas (sin portada)	
Vídeo demostración (opcional)	Enlace YouTube/Drive, máx. 3 min	

Nota: el vídeo de demostración es opcional.

7. Criterios de Evaluación – Rúbrica Detallada

La siguiente rúbrica se usará para corregir el proyecto. Léela con atención antes de empezar: cada fila describe exactamente lo que se espera para obtener la puntuación máxima en ese criterio.

Criterio de Evaluación	Indicadores de Logro	Puntos máx.	Peso (%)	Cómo se valora
1. Configuración del entorno y del robot	– Escenario cargado correctamente en Webots	**1,5**	15%	*0 pts: no arranca* *0,75 pts: arran-*

Criterio de Evaluación	Indicadores de Logro	Puntos máx.	Peso (%)	Cómo se valora
	– Robot con sensores y actuadores declarados en el controlador – Código compila y se ejecuta sin errores fatales			*ca con errores menores* *1,5 pts: todo correcto*
2. Lectura de sensores	– Sensores de distancia (IR o sonar) inicializados – Valores leídos correctamente en cada paso de simulación – Se muestran/usan valores en consola o lógica de control	**1,5**	15%	*0 pts: sin lectura* *0,75 pts: lectura parcial o incorrecta* *1,5 pts: lectura correcta y usada*
3. Control de movimiento básico	– El robot se mueve hacia adelante – Puede girar izquierda/derecha de forma controlada – Velocidades ajustadas de manera proporcional o razonada	**1,5**	15%	*0 pts: inmóvil* *0,75 pts: solo avanza* *1,5 pts: avanza y gira correctamente*
4. Comportamiento autónomo (evitar obstáculos)	– El robot detecta obstáculos antes de colisionar – Implementa al menos una estrategia de evasión (girar, retroceder...) – El robot navega durante al menos 60 s sin colisión grave	**2,0**	20%	*0 pts: choca constantemente* *1 pt: evita algunos* *2 pts: evita correctamente 60 s*
5. Calidad y legibilidad del código	– Variables con nombres descriptivos – Funciones bien delimitadas (al menos 2) – Comentarios que explican la lógica principal	**1,5**	15%	*0 pts: código ilegible* *0,75 pts: parcialmente organizado* *1,5 pts: claro, comentado y estructurado*
6. Memoria / Informe escrito	– Describe el problema y la solución adoptada – Incluye capturas de la simulación funcionando – Reflexiona sobre dificultades y posibles mejoras	**1,5**	15%	*0 pts: no entregado* *0,75 pts: incompleto* *1,5 pts: completo y reflexivo*
7. Defensa oral (presentación breve)	– Explica qué hace el robot y cómo funciona su controlador – Responde preguntas del profesor con fundamento – Demuestra comprensión real del código entregado	**0,5**	5%	*0 pts: no defiende* *0,25 pts: respuestas vagas* *0,5 pts: dominio claro*
	TOTAL	**10**	**100%**	

Escala de calificación cualitativa

0 – 4,9	5 – 5,9	6 – 6,9	7 – 8,9	9 – 10
Insuficiente	Suficiente	Bien	Notable	Excelente

8. Recomendaciones Finales

Planificación sugerida (2 - 3 semanas)

Semana	Actividades recomendadas
1	Instalar Webots y abrir el proyecto facilitado. Completar Apartado A (inicialización) y Apartado B (sensores). Verificar en consola que los sensores funcionan.
2	Implementar Apartado C (movimiento) y Apartado D (evasión). Realizar pruebas y ajustar umbrales. Empezar a tomar notas para el informe.
3	Refactorizar el código (Apartado E), completar el informe y preparar la defensa oral. Graba el vídeo opcional si lo deseas.